

NeurIPS 2023 | Teaching RL Agents to Borrow, Mix, and Swap Skills at Will

UC San Diego and UC Santa Barbara introduce KGRL and KIAN, an attention-based actor that turns external knowledge into modular, reorderable building blocks

When a person learns to ride a motorcycle, they don't start from nothing. They borrow balance from a skateboard, pedaling-and-steering from a bicycle, and stitch those habits together into something new. Reinforcement learning (RL) agents rarely get to work this way. Even on tasks a human would pick up in minutes, an agent may need millions of environment interactions, because it has no easy way to reuse skills it, or others, already know.

Researchers have tried to close this gap by injecting external knowledge policies into RL: hand-written rules, pretrained controllers, or behavior-cloned experts that nudge the agent toward good actions. It helps, but most of these methods bake the knowledge in rigidly. The fusion mechanism depends on how many policies there are and the order they appear in, so adding a new skill, dropping a useless one, or rearranging the set means changing the architecture or retraining large parts of the model. The knowledge is there, but it is not modular.

A NeurIPS 2023 paper from UC San Diego and UC Santa Barbara attacks exactly this rigidity. It proposes Knowledge-Grounded Reinforcement Learning (KGRL), a paradigm for fusing many external knowledge policies with an agent's own learnable policy, and the Knowledge-Inclusive Attention Network (KIAN), an actor architecture that lets those policies be freely rearranged, added, or replaced without retraining the rest of the network. Along the way, the authors identify and fix an exploration pathology they call entropy imbalance.

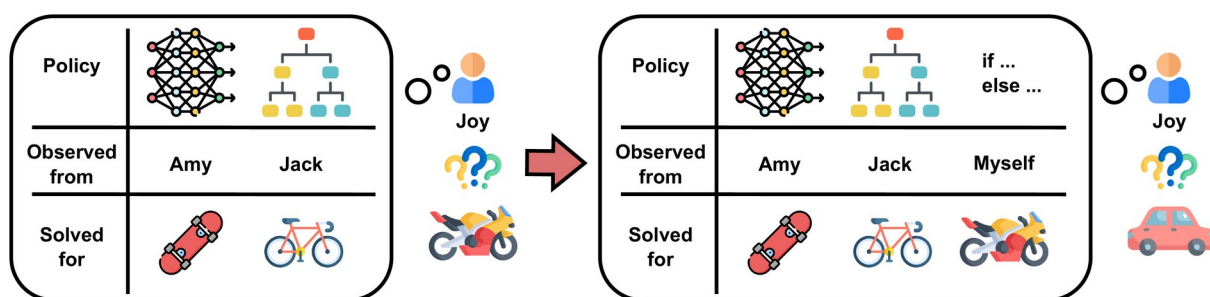


Figure 1. The knowledge-acquirable, compositional, and incremental properties KGRL aims for. Joy first learns to ride a motorcycle by observing Amy skateboarding and Jack biking; later she learns to drive a car with the knowledge set expanded by her own developed motorcycling strategy. Skills are observed, reused, and added on the fly.

Paper: <https://arxiv.org/abs/2210.03729>

Code: <https://github.com/Pascalson/KGRL.git>

The problem: knowledge that can't be rearranged

The authors frame efficient, human-like learning as five properties: it should be knowledge-acquirable (able to take in external strategies), sample-efficient, generalizable to new tasks, compositional (able to combine skills), and incremental (able to grow its skill set over time). Existing knowledge-guided RL methods satisfy some of these but stumble on flexibility.

Methods like KoGuN and A2T fuse external policies through mechanisms tied to a fixed input ordering or to per-policy parameters. That coupling is the crux of the problem: because each policy's contribution is entangled with the number and position of the others, you cannot swap one out or slot a new one in without rebuilding the fusion module and retraining. A single trained agent therefore can't carry its knowledge cleanly into a new task, which is the opposite of how a person reuses skills.

KIAN: encoding every policy as an attention key

KGRL keeps the agent's own policy, an inner actor that can always learn a solution from scratch, and fuses it with a set of external knowledge policies. The key design question is how to combine them so the set stays modular. KIAN's answer is to describe each policy not by its position in a list, but by a learnable embedding vector, which the paper calls that policy's key and treats as its identity.

KIAN has three parts. The inner actor is an ordinary learnable policy, so even if every external policy is useless for the current task, the agent can still discover its own solution. Each knowledge policy, including the inner one, gets a learnable key that represents the whole policy, independent of any particular state or action. A state-dependent query is then produced from the observation and compared against every key by a dot product; a softmax over those scores yields attention weights, and the final policy is a weighted sum of all the policies.

Concretely, the query $\Phi(s_t)$ is dotted with each key (k_{in} for the inner actor, k_{gj} for knowledge policy j), normalized, and passed through a softmax to produce fusion weights; the acted policy is $\pi(\cdot|s_t) = \hat{w}_{in} \cdot \pi_{in}(\cdot|s_t) + \sum_j \hat{w}_{gj} \cdot \pi_{gj}(\cdot|s_t)$. Because each key lives in a shared embedding space and stands on its own, the policies are unordered. Adding a policy means adding a key; removing one means deleting its key; replacing one means swapping its key, with no changes anywhere else in KIAN. For continuous control, one policy is selected via Gumbel-softmax and sampled with the reparameterization trick, keeping the whole actor differentiable end to end.

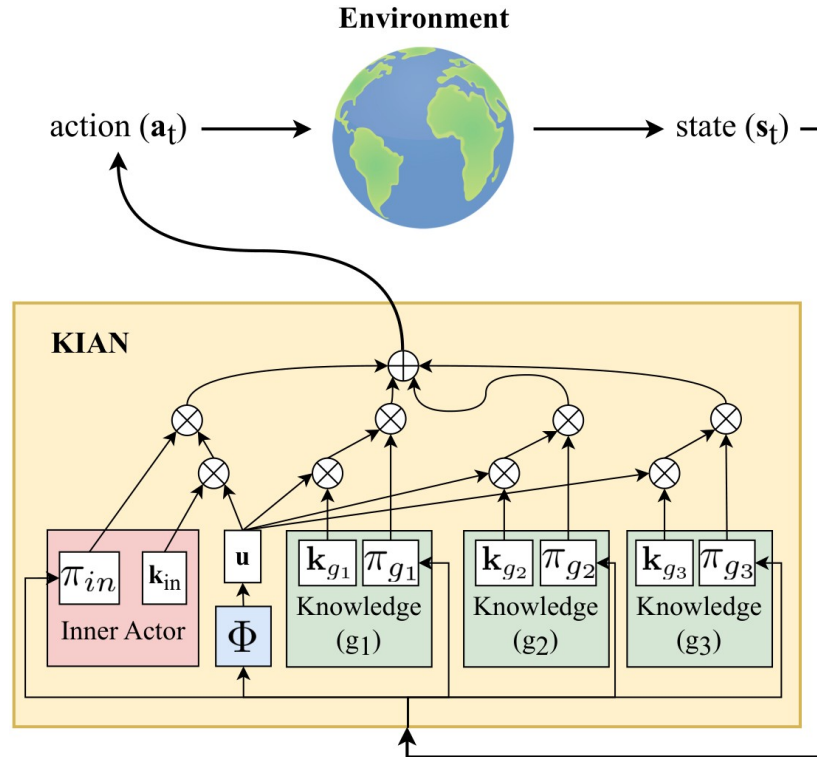


Figure 2. The KIAN architecture. The inner actor develops the agent's own strategy while external knowledge policies g_1, g_2, g_3 are fused through embedding-based attentive action prediction. Each policy is addressed by its own key, so knowledge can be freely rearranged, added, or removed without touching the rest of the network.

Fixing entropy imbalance

The paper's second technical contribution is subtler. In maximum-entropy RL, used for exploration in methods like SAC, the agent maximizes the entropy of its action distribution. When that distribution is a fusion of several policies, the authors show the entropy term can push the weights unevenly across policies, over-favoring some and starving others of exploration. They call this entropy imbalance, prove the conditions under which it arises, and propose modified policy distributions that rebalance the objective so exploration stays healthy when many policies are fused.

Results: the only method that works everywhere

The agent is tested on two benchmark suites: MiniGrid, a family of discrete grid-world tasks of increasing difficulty (Empty-Random, Unlock, DoorKey, Dynamic-Obstacles, LavaCrossing, MultiRoom, KeyCorridor), and OpenAI-Robotics, continuous-control manipulation (Push, Slide, Pick-and-Place, Reach). Crucially, every method starts from the same initial knowledge set built from deliberately sub-optimal if-else programs, simple routines like 'pick up a key' or 'move toward the goal' that cannot solve any task on their own. All runs use ten random seeds and report 95% confidence intervals.

Given that sub-optimal knowledge, KIAN is the only method that succeeds across every environment, and it is the most sample-efficient, its advantage widening as tasks get harder. On the hardest grid tasks such as MultiRoom-N4-S5 and KeyCorridorS3R2, the baselines (BC, RL, RL+BC, KoGuN, A2T) barely move off the floor while KIAN climbs steadily. KIAN also produces lower-variance learning curves, and it stays stable in continuous control where A2T, which ignores entropy imbalance, largely collapses.

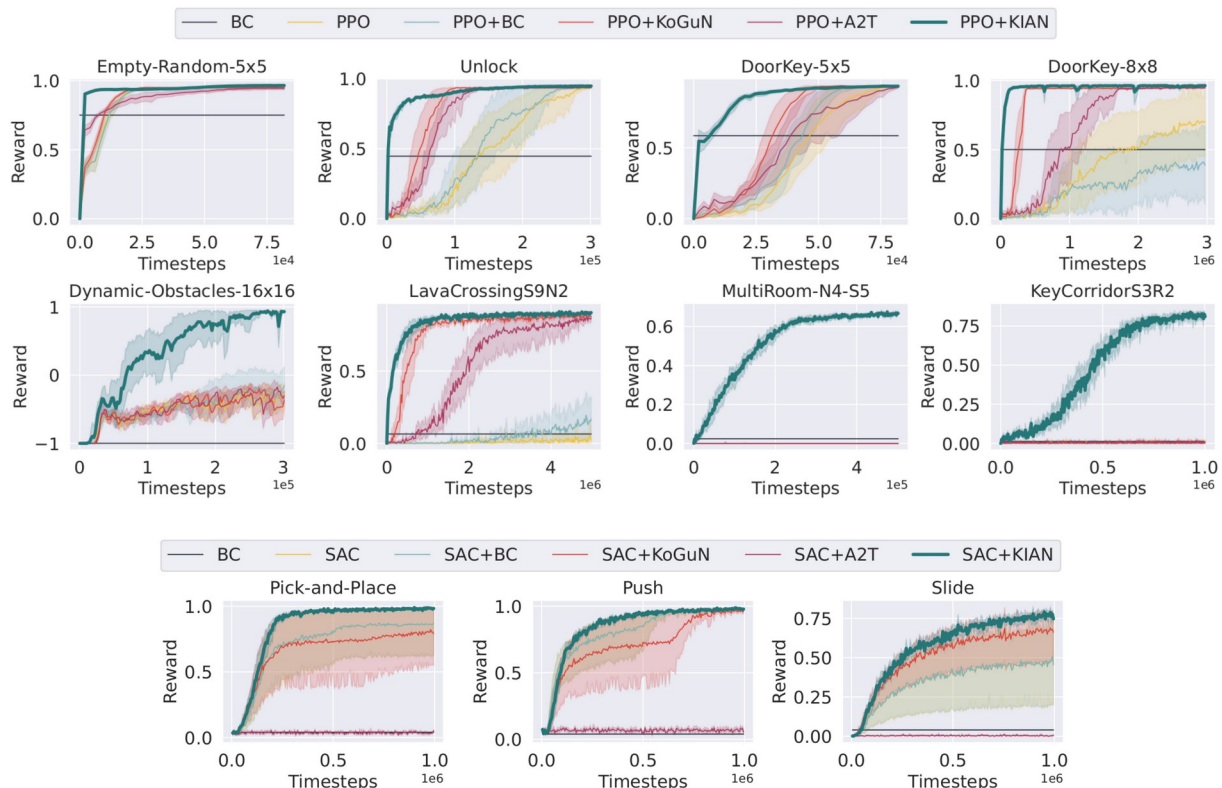


Figure 3. Sample-efficiency learning curves on MiniGrid (top two rows) and OpenAI-Robotics (bottom row). Starting from a knowledge set that cannot complete a task on its own (see the flat BC line), KIAN (thick teal) reaches high reward with far fewer samples across all tasks, and its lead grows on the harder environments.

Transfer: carrying knowledge to new tasks

Because KIAN's knowledge set is modular, a trained agent can be dropped into a different task zero-shot. In simple-to-complex (S2C) transfer, train on a small environment, test on a larger one, and complex-to-simple (C2S) transfer, KIAN outperforms every baseline on most transfers, and with noticeably smaller variance. A few headline numbers make the gap concrete.

Table 1. Zero-shot transfer reward (higher is better): KIAN versus the strongest baseline in each setting.

Zero-shot transfer setting	KIAN	Best baseline
Empty-Random 5x5 → 16x16 (S2C)	0.93	0.53 (KoGuN)
Pick-and-Place, 10 times goal range (S2C)	0.72	0.30 (RL+BC)
DoorKey-8x8 → Reach (C2S)	1.00	0.80 (RL / RL+BC)

Ablation: the entropy fix earns its place

How much does the entropy-imbalance fix actually matter? The ablation runs KIAN with and without the modified policy distributions. Without them, performance drops sharply on the demanding tasks, Dynamic-Obstacles-16x16, MultiRoom-N4-S5, and the continuous Pick-and-Place, Push, and Slide, where an agent maximizing entropy collapses onto a single policy and stops exploring effectively. Turning the modifications on restores efficient exploration and recovers strong performance, confirming the fix is essential rather than cosmetic.

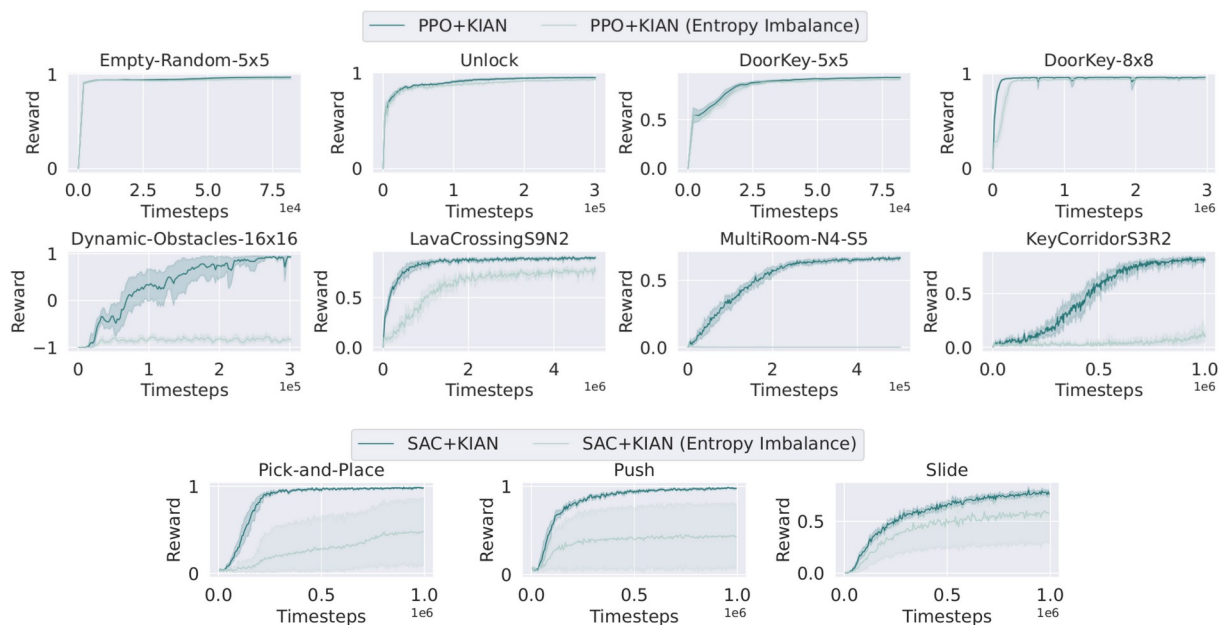


Figure 5. KIAN with and without the entropy-imbalance fix. The gap is small on easy tasks but large on hard ones (Dynamic-Obstacles-16x16, MultiRoom-N4-S5, and the robotics tasks), where the fix restores efficient exploration and the unfixed variant stalls.

Finally, dedicated compositional and incremental experiments show the modular design paying off directly: KIAN reuses its previously learned keys and inner policy to acquire two tasks in sequence with fewer total samples than learning each from scratch, the concrete, measurable version of Joy adding motorcycling to her skill set before learning to drive.

Takeaway

The lasting idea is simple: treat each knowledge policy as an independent, attention-addressable key, and external knowledge becomes a set of modular building blocks rather than a fixed, entangled module. An agent can add, drop, or reorder its policies at any time without retraining, and the entropy-imbalance fix keeps exploration efficient once many policies are fused. The result is an RL actor that learns faster, generalizes better, and stays flexible.

The honest boundaries are worth keeping in view. The gains are demonstrated on grid-world and simulated-robotics benchmarks with knowledge sets built from simple programmatic policies; how the approach scales to richer, higher-dimensional knowledge sources and real-robot settings is left for future work. But as a step toward agents that reuse and recombine skills the way people do, KIAN's key-based modularity is a clean and practical one.